

Week - 9 Programming Assignments

Graphs Part 2

For the assignments you submit, provide a solution to the problems by writing a function that will be called inside main () function. The logic for the given problem must be implemented inside the function. If no functions are given, or the logic is implemented inside the main function, points will be **DEDUCTED**.

To submit this assignment, all students should group all C-script files in '.zip' format with the zip file formatted as:

STUDENT_ID_ASSIGNMENT_ID.zip

Ex: **68xxxxx3401_01.zip**

ALL C-script files should also be formatted as:

ASSIGNMENT_ID_QUESTION_ID_STUDENT_ID.c

Ex: **01_01_3401.c**

Regarding help, you may contact the TAs for assistance(at office hours).

Lists of Things to check before submitting:

- Name of the Files submitted
- Format of the Files submitted
- Sending the correct C files

Problem 1: Matrix Manipulation 101

$\{1, 2, 3\}$
 $\{4, 5, 6\}$

Given a Matrix ($\{7, 8, 9\}$), write multiple functions to find:

- The **Diagonal** Sum ($\backslash$) of the matrix
- The **Row-wise** Sum ($-$) of the matrix
- The **Column-wise** Sum ($|$) of the matrix
- & The **number** of the row with the **maximum** sum

Rules/Constraints:

- The Matrix is initialized inside the code. (No need for inputs)
- The Matrix is 0-indexed (first row or column will be '0')
- The Diagonal sum is from left to right only ($\backslash$)

Example 1:

|Input: (None, the matrix is already set in the main program)

|Output:

15
6 15 24
12 15 18
2

Explanations:

Diagonal	=	1+5+9 (15)		
Row-wise	=	1+2+3 (6)	4+5+6 (15)	7+8+9 (24)
Column-wise	=	1+4+7 (12)	2+5+8 (15)	3+6+9 (18)
Max-sum row	=	2 (0-based index and 24 is the max sum)		

Here's the Code:

```
#include <stdio.h>

int diagonal_sum(int matrix[3][3]){
    int sum = 0;
    ?
    return sum;
}

int row_sum(int matrix[3][3], int row){
    int sum = 0;
    ?
    return sum;
}

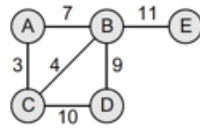
int column_sum(int matrix[3][3], int col){
    int sum = 0;
    ?
    return sum;
}

int max_row_sum(int matrix[3][3]){
    int max_sum = 0;
    int max_row = 0;
    ?
    return max_row;
}

int main()
{
    int matrix[3][3] = {
        {1,2,3},
        {4,5,6},
        {7,8,9}
    };
    int sum = diagonal_sum(matrix);
    printf("%d\n", sum);
    for(int i=0;i<3;i++)
    {
        sum = row_sum(matrix,i);
        printf("%d ", sum);
    }
    printf("\n");
    for(int i=0;i<3;i++)
    {
        ?
    }
    printf("\n");
    sum = max_row_sum(matrix);
    printf("%d", sum);
    return 0;
}
```

[For Template use]

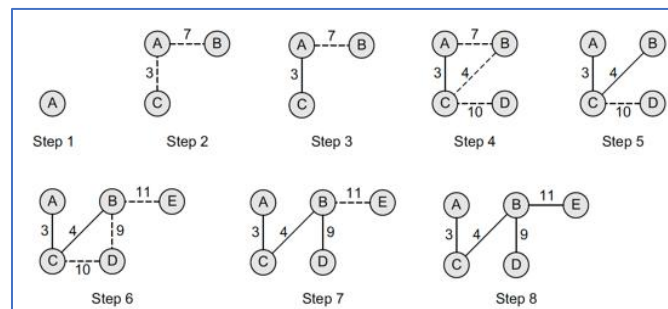
Problem 2: Prim and Kruskal's



Given a Graph (), Write a function either called Prim() or Kruskal() to implement a minimum spanning tree searching algorithm. You will need to write **TWO** files, one using Prim's and one using Kruskal's.

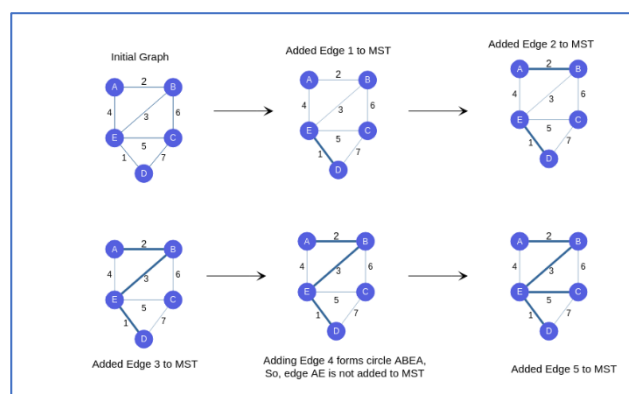
Rules/Constraints:

- The graph is already initialized in the code.
- The **ONLY** input is the **STARTING** vertex.
- The Algorithm of Prim's works as follows:
 - Visited array to record visited vectors/nodes.
 - Attempt to span the tree from already visited nodes.
 - Span out using the edge with the smallest number/weight.
 - Print out the smallest edge to be adding to the minimum spanning tree.
 - Repeat the previous steps until every node is visited.



[Prim Example minimum spanning tree]

- The Algorithm of Kruskal's works as follows:
 - Add the edge with the lowest weight to the Minimum Spanning Tree (MST).
 - DO NOT add an edge that makes the MST a cycle (the graph loops back)
 - Repeat until all vectors/nodes are connected.



Example 1(Prim's):

|Input:

0

REMINDER: 0 is A. As such, this is starting at vertex A.

|Output:

A - C: 3

C - B: 4

B - D: 9

B - E: 11

REMINDER: Use `printf("%c ",vertex+65)` to print as the Alphabet.

Example 2(Prim's):

|Input:

3

|Output:

D - B: 9

B - C: 4

C - A: 3

B - E: 11

Example 1(Kruskal's):

|Input:

0

|Output:

A - C: 3

C - B: 4

B - D: 9

B - E: 11

REMINDER: As Kruskal's always start from the lowest edge, it will always have the same output shown here.

Here's the Code:

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 10
#define MAX_INT INT_MAX

typedef struct Graph_t{
    int V;
    int adj[MAX_VERTICES][MAX_VERTICES];
} Graph;

Graph *Graph_create(int V){}
void Graph_destroy(Graph *g){}
void Graph_addEdge(Graph *g, int src, int dst,int weight){
    g->adj[src][dst] = weight;
    ? //as an Undirect graph, you need an additional line of code here
}

void Prim(Graph *g,int s){
    bool visited[MAX_VERTICES];
    for (int i = 0; i < g->V; i++) {
        visited[i] = false;
    }
    visited[s] = true;
    for (int i = 0; i < g->V-1; i++) {
        //Implementation here
        ?//Step 1: Which nodes do you look for? (Hint:Use your visited[])
        for (int j = 0; j < g->V; j++){
            //Step 2: Checking Neighbors (Remember last session, do you look for visited or unvisited?)
            ?
            //Step 3: How would you compare the weight? (Store the edge with the smallest weight)
            ?
        }
    }
}

int main()
{
    int v = 5;
    Graph *g = Graph_create(v); //A=0,B=1,C=2,D=3,E=4
    Graph_addEdge(g, 0, 1, 7);
    Graph_addEdge(g, 0, 2, 3);
    Graph_addEdge(g, 1, 2, 4);
    Graph_addEdge(g, 1, 3, 9);
    Graph_addEdge(g, 1, 4, 11);
    Graph_addEdge(g, 2, 3, 10);

    int starting_v;
    scanf("%d",&starting_v);
    Prim(g,starting_v);

    Graph_destroy(g);
    return 0;
}
```

[This code uses Prim's]

```

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 10

typedef struct {
    int src, dst, weight;
} Edge;

typedef struct Graph_t {
    int V;
    int adj[MAX_VERTICES][MAX_VERTICES];
} Graph;

Graph *Graph_create(int V){}
void Graph_destroy(Graph *g){}
void Graph_addEdge(Graph *g, int src, int dst, int weight){}

int find(int parent[], int i){
    if (parent[i] != i)
        parent[i] = find(parent, parent[i]); //recursion to find parent
    return parent[i];
}

void unite(int parent[], int rank[], int x, int y){
    int rx = find(parent, x);
    int ry = find(parent, y);
    if (rx != ry) {
        //Check which rank is larger
        //node with lower rank with become child of the other
        //else source(x) becomes parent of dest(y), then increase rank of (x)
    }
}

int compare(const void *a, const void *b){
    return ((Edge *)a)->weight - ((Edge *)b)->weight;
}

void Kruskal(Graph *g){
    Edge edges[MAX_VERTICES * MAX_VERTICES];
    int edgeCount = 0;
    //Extract edges (avoid duplicates)
    for (int i = 0; i < g->V; i++) {
        for (int j = i + 1; j < g->V; j++) {
            //Extract adj[i][j] to become edge
        }
    }
    qsort(edges, edgeCount, sizeof(Edge), compare);
    int parent[MAX_VERTICES];
    int rank[MAX_VERTICES];

    for (int i = 0; i < edgeCount && mstEdges < g->V - 1; i++) {
        Edge e = edges[i];
    }
}

```

```

        //Check for loops
        //Printf edge
        //Add the edge to the 'mst' using unite
        ?
        mstEdges++;
    }
}

int main()
{
    int v = 5;
    Graph *g = Graph_create(v); // A=0,B=1,C=2,D=3,E=4
    Graph_addEdge(g, 0, 1, 7);
    Graph_addEdge(g, 0, 2, 3);
    Graph_addEdge(g, 1, 2, 4);
    Graph_addEdge(g, 1, 3, 9);
    Graph_addEdge(g, 1, 4, 11);
    Graph_addEdge(g, 2, 3, 10);

    Kruskal(g);

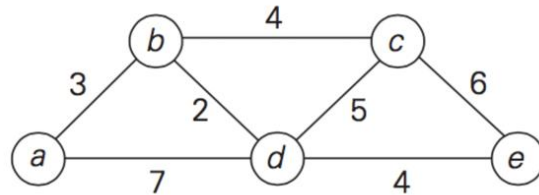
    Graph_destroy(g);
    return 0;
}

```

[This code uses Kruskal's]

Problem 3: Dijkstra in the House

Write a function `Dijkstra()` for the purpose of implementing Dijkstra's Algorithm to traverse a given undirected weighted graph.



Rules/Constraints:

- The graph is already initialized in the code.
- The **ONLY** inputs are the **STARTING** vertex and the **DESTINATION**.
- The Algorithm of Dijkstra's works as follows:
 - Create a visited, distance and parent array
 - **Visited:** to record already visited nodes
 - **Distance:** to record the shortest distance to that node
 - **Parent:** to record the nodes in the path needed to get to the destination node
 - Start from starting/source vertex(distance 0)
 - **[From here, find the next unvisited node with the smallest distance.**
 - Update the neighbor nodes of the current(previously unvisited) with their smallest distance.
 - Add current node as parents of their neighbor **nodes]**
 - Repeat the steps in the square brackets until all reachable nodes have their smallest distance
- The Outputs are:
 - The **Path** needed to reach the destination
 - The **Distance** of the path

Example 1:

|Input:

0 2

REMINDER: 0 is A, 2 is C. As such, this is starting at vertex A and the Destination is vertex C

|Output:

A B C

7

Example 2:

|Input:

4 0
|Output:
E D B A
9

Here's the Code:

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 10
#define INF INT_MAX

typedef struct Graph_t
{
    int V;
    int adj[MAX_VERTICES][MAX_VERTICES];
} Graph;

Graph *Graph_create(int V){}
void Graph_destroy(Graph *g){}
void Graph_addEdge(Graph *g, int src, int dst,int weight){} //Undirected Graph
still

void dijkstra(Graph *g, int src, int dst){
    int dist[MAX_VERTICES];
    int parent[MAX_VERTICES];
    bool visited[MAX_VERTICES] = {false};
    ?
    }
}
?
}

void printPath(int parent[], int j){
    if (parent[j] == -1){
        printf("%c ", j+65);
        return;
    }
    printPath(parent, parent[j]);
    printf("%c ", j+65);
}

int minDistance(int dist[], bool visited[], int v){
    int min = INF, min_index = -1;
    ?
    return min_index;
}
```

```

int main()
{
    int v = 5;
    Graph *g = Graph_create(v); //A=0,B=1,C=2,D=3,E=4
    Graph_addEdge(g, 0, 1, 3); //A-B 3
    Graph_addEdge(g, 0, 3, 7); //A-D 7
    Graph_addEdge(g, 1, 2, 4); //B-C 4
    Graph_addEdge(g, 1, 3, 2); //B-D 2
    Graph_addEdge(g, 2, 3, 5); //C-D 5
    Graph_addEdge(g, 2, 4, 6); //C-E 6
    Graph_addEdge(g, 3, 4, 4); //D-E 4

    int starting_v,dst;
    scanf("%d %d",&starting_v,&dst);
    dijkstra(g, starting_v, dst);
    Graph_destroy(g);
    return 0;
}

```

[This code is for template use]